Sakarya Üniversitesi

Bilgisayar ve Bilişim Bilimleri Fakültesi

Bilgisayar Mühendisliği Anabilim Dalı

Bulut Bilişim Ödevi

NATS: Cloud Native Mesajlaşma Sisteminin Teknik Değerlendirmesi

CNCF Graduated Proje Analizi

Ders:	Bulut Bilişim
Dönem:	2025–2026 Bahar
Seviye:	Yüksek Lisans
Öğretim Üyesi:	Dr. Öğr. Üyesi Baran Kaynak
Seçenek:	B — CNCF Projesi Keşfi ve Teknik Değerlendirme
Seçilen Proje:	NATS (Graduated Statüsü)
Teslim Tarihi:	19 Nisan 2026

Sakarya, Nisan 2026

İçindekiler

1 Giriş ve Motivasyon **3**
1.1 Bulut-Yerel Ekosisteminde Mesajlaşma Sorunu . . . 3
1.2 NATS Nedir? . . . 3
1.3 Proje Seçim Gerekçesi . . . 3

2 Mimari Analiz **4**
2.1 NATS'ın İç Mimarisi . . . 4
2.2 İletişim Modelleri . . . 4
2.2.1 Publish/Subscribe (Core NATS) . . . 4
2.2.2 Request/Reply . . . 4
2.2.3 JetStream (Persistent Messaging) . . . 4
2.3 Subject Sistemi . . . 5
2.4 Tasarım Felsefesi . . . 5

3 Kurulum ve Deneyim **5**
3.1 Ortam Bilgileri . . . 6
3.2 Kurulum Adımları . . . 6
3.2.1 Adım 1: Docker Kurulumu . . . 6
3.2.2 Adım 2: Proje Yapısı . . . 7
3.2.3 Adım 3: docker-compose.yml Hazırlama . . . 7
3.2.4 Adım 4: Servislerin Başlatılması . . . 7
3.3 Test 1: Publish/Subscribe . . . 8
3.4 Test 2: JetStream (Kalıcı Mesajlaşma) . . . 9
3.5 Test 3: Monitoring API . . . 10
3.6 Test 4: Consumer (Tüketici) Oluşturma . . . 11
3.7 Karşılaşılan Sorunlar ve Çözümler . . . 13

4 Karşılaştırmalı Değerlendirme **13**
4.1 Genel Karşılaştırma . . . 13
4.2 Performans Analizi . . . 13
4.3 Topluluk ve Ekosistem . . . 14
4.4 Öğrenme Eğrisi . . . 14
4.5 Kullanım Senaryosu Uyumluluğu . . . 14

5 Sonuç ve Öneriler **15**
5.1 Güçlü Yönler . . . 15
5.2 Zayıf Yönler . . . 15

5.3 Hangi Senaryolarda NATS Tercih Edilmeli? 15
5.4 Öğrenilen Dersler 15

Kaynakça **16**

1. Giriş ve Motivasyon

1.1. Bulut-Yerel Ekosisteminde Mesajlaşma Sorunu

Modern bulut-yerel mimarilerde, onlarca hatta yüzlerce bağımsız mikro hizmet birbiriyle sürekli iletişim kurmaktadır. Bu iletişimin senkron yöntemlerle (REST, gRPC) sağlanması; gecikme birikimi (*latency accumulation*), kırılgan bağımlılıklar (*tight coupling*) ve ölçeklendirme zorluğu gibi ciddi sorunlara yol açmaktadır.

Asenkron mesajlaşma sistemleri bu sorunlara yapısal bir çözüm sunmaktadır. Ancak mevcut çözümlerin büyük çoğunluğu ya aşırı karmaşık altyapı gereksinimleri (Apache Kafka'nın ZooKeeper/KRaft bağımlılığı), ya yüksek bellek tüketimi (RabbitMQ), ya da sınırlı protokol desteği gibi dezavantajlar taşımaktadır.

Temel Problem

N adet mikro hizmetin birbirleriyle doğrudan iletişim kurduğu bir sistemde bağlantı sayısı $\frac{N(N-1)}{2}$ olur. 10 servis için bu 45, 50 servis için 1225 bağlantı anlamına gelir. NATS, tüm bu bağlantıları tek bir merkezi *hub* üzerinden yöneterek bu karmaşıklığı N bağlantıya indirger.

1.2. NATS Nedir?

NATS, açık kaynaklı, yüksek performanslı bir mesajlaşma sistemidir. İlk olarak 2010 yılında Derek Collison tarafından Go programlama dili ile geliştirilmiş ve 2016 yılında Cloud Native Computing Foundation (CNCF) bünyesine alınmıştır. 2022 yılında CNCF **Graduated** statüsüne ulaşarak üretim ortamı olgunluğunu kanıtlamıştır [4].

Temel değer önerisi: NATS, tek bir binary dosyasıyla çalışan, Zookeeper veya benzeri harici bağımlılıklar gerektirmeyen, pub/sub, request/reply ve JetStream (persistent streaming) gibi birden fazla iletişim modelini destekleyen hafif bir mesajlaşma katmanıdır.

1.3. Proje Seçim Gerekçesi

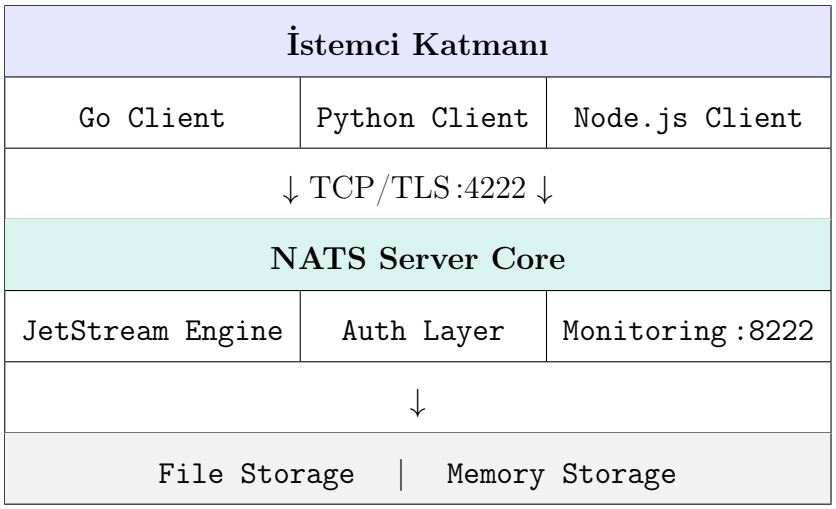
Bu çalışmada NATS'ın seçilmesinin başlıca nedenleri şunlardır:

- IoT ve endüstriyel sistemlerde düşük gecikmeli mesajlaşma ihtiyacı
- Mikro hizmet mimarilerinde servisler arası iletişim gereksinimleri
- Tek container ile çalışabilmesi - kısıtlı kaynaklarda dahi test edilebilir olması
- CNCF Graduated statüsü - uzun vadeli destek güvencesi
- ~10 MB RAM tüketimi ile rakipsiz kaynak verimliliği

2. Mimari Analiz

2.1. NATS'ın İç Mimarisi

NATS'ın mimarisi üç ana katmandan oluşmaktadır: istemci katmanı, çekirdek sunucu katmanı ve depolama katmanı [1].



Şekil 1: NATS Sistem Mimarisi

2.2. İletişim Modelleri

NATS üç temel iletişim modelini desteklemektedir [2]:

2.2.1. Publish/Subscribe (Core NATS)

En temel modeldir. Publisher bir konuya (*subject*) mesaj gönderir; o konuya abone olan tüm subscriber'lar mesajı alır. Mesaj kalıcı değildir — abone bağlı değilse mesaj kaybolur.

$$\texttt{Publisher} \xrightarrow{\text{orders.new}} \textbf{NATS Server} \xrightarrow{\text{fan-out}} \texttt{Subscriber1, Subscriber2, ...}$$

2.2.2. Request/Reply

Senkron benzeri çift yönlü iletişim için kullanılır. Publisher bir yanıt bekler; ilgili servis yanıtı gönderir. Mikro hizmetler arası RPC benzeri iletişim için idealdir.

2.2.3. JetStream (Persistent Messaging)

NATS 2.2 ile eklenen bu özellik, Kafka benzeri kalıcı mesajlaşmayı NATS içinde sağlar. Mesajlar stream'lerde saklanır; consumer'lar geç bağlansa dahi mesajları alabilir [3].

Tablo 1: İletişim Modellerinin Karşılaştırması

Özellik	**Pub/Sub**	**Request/Reply**	**JetStream**
Kalıcılık	Hayır	Hayır	Evet
Yön	Tek yön	Çift yön	Tek yön
Teslimat garantisi	En fazla bir kez	En fazla bir kez	En az / tam bir kez
Gecikme	<1ms	<2ms	<5ms
Kullanım alanı	Bildirim, yayın	RPC, sorgulama	Sipariş, ödeme

2.3. Subject Sistemi

NATS'ta tüm mesajlaşma **subject** (konu) bazında gerçekleşir. Subject'ler nokta ile ayrılan hiyerarşik yapıdadır:

Tablo 2: NATS Subject Wildcard Örnekleri

Subject	**Tür**	**Eşleşme Örneği**
`orders.new`	Tam eşleşme	Yalnızca `orders.new`
`orders.*`	Tek seviye wildcard	`orders.new`, `orders.cancel`
`orders.>`	Çok seviyeli wildcard	`orders.new.usa`, `orders.items.list`

2.4. Tasarım Felsefesi

NATS'ın tasarımında öne çıkan prensipler şunlardır:

- **Basitlik:** Tek binary, sıfır harici bağımlılık
- **Performans önceliği:** Mesaj yönlendirme için hashmap tabanlı O(1) subject matching
- **Güvenlik:** TLS, token/NKey/JWT tabanlı kimlik doğrulama
- **Multi-tenancy:** Account sistemi ile tam izolasyon
- **Dayanıklılık:** JetStream ile at-least-once ve exactly-once teslimat garantisi

3. Kurulum ve Deneyim

3.1. Ortam Bilgileri

Tablo 3: Test Ortamı Özellikleri

Parametre	Değer
Sunucu İşletim Sistemi	Ubuntu 22.04.5 LTS
RAM	16 GB
CPU	6 Core
Depolama	96 GB SSD
Docker Sürümü	29.1.3
Docker Compose Sürümü	1.29.2
NATS Sürümü	2.12.6 (Go 1.25.8)
Bağlantı	SSH (Uzak VPS)
IP Adresi	51.38.113.222

3.2. Kurulum Adımları

3.2.1. Adım 1: Docker Kurulumu

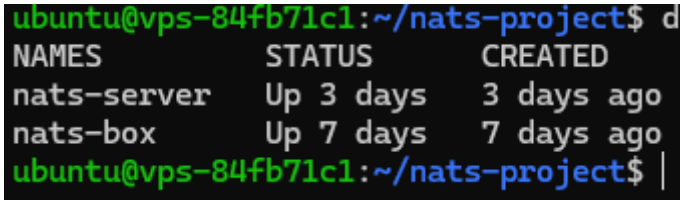
```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io docker-compose
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
newgrp docker
```

Listing 1: Docker ve Docker Compose Kurulumu

Doğrulama çıktısı:

```
Docker version 29.1.3, build 29.1.3-0ubuntu3~22.04.1
docker-compose version 1.29.2, build unknown
```

Listing 2: Sürüm Doğrulama



Şekil 2: Docker Konteynerlerinin Çalışır Durumu

3.2.2. Adım 2: Proje Yapısı

```
mkdir ~/nats-project && cd ~/nats-project
```

Listing 3: Proje Klasörü Oluşturma

3.2.3. Adım 3: docker-compose.yml Hazırlama

```yaml
version: '3.8'

services:
  nats:
    image: nats:latest
    container_name: nats-server
    ports:
      - "4222:4222"   # Client connections
      - "8222:8222"   # HTTP Monitoring
      - "6222:6222"   # Cluster routing
    command: ["-m", "8222", "-js"]
    restart: unless-stopped

  nats-box:
    image: natsio/nats-box:latest
    container_name: nats-box
    depends_on:
      - nats
    stdin_open: true
    tty: true
    restart: unless-stopped
```

Listing 4: docker-compose.yml İçeriği

Önemli Not

`-js` flag'i JetStream özelliğini aktifleştirir. `-m 8222` ise HTTP monitoring endpoint'ini açar. `nats-box`, NATS'ın resmi test ve hata ayıklama istemcisidir.

3.2.4. Adım 4: Servislerin Başlatılması

```
docker-compose up -d
# Cikti:
# Creating nats-server ... done
# Creating nats-box    ... done
```

Listing 5: Docker Compose ile Başlatma

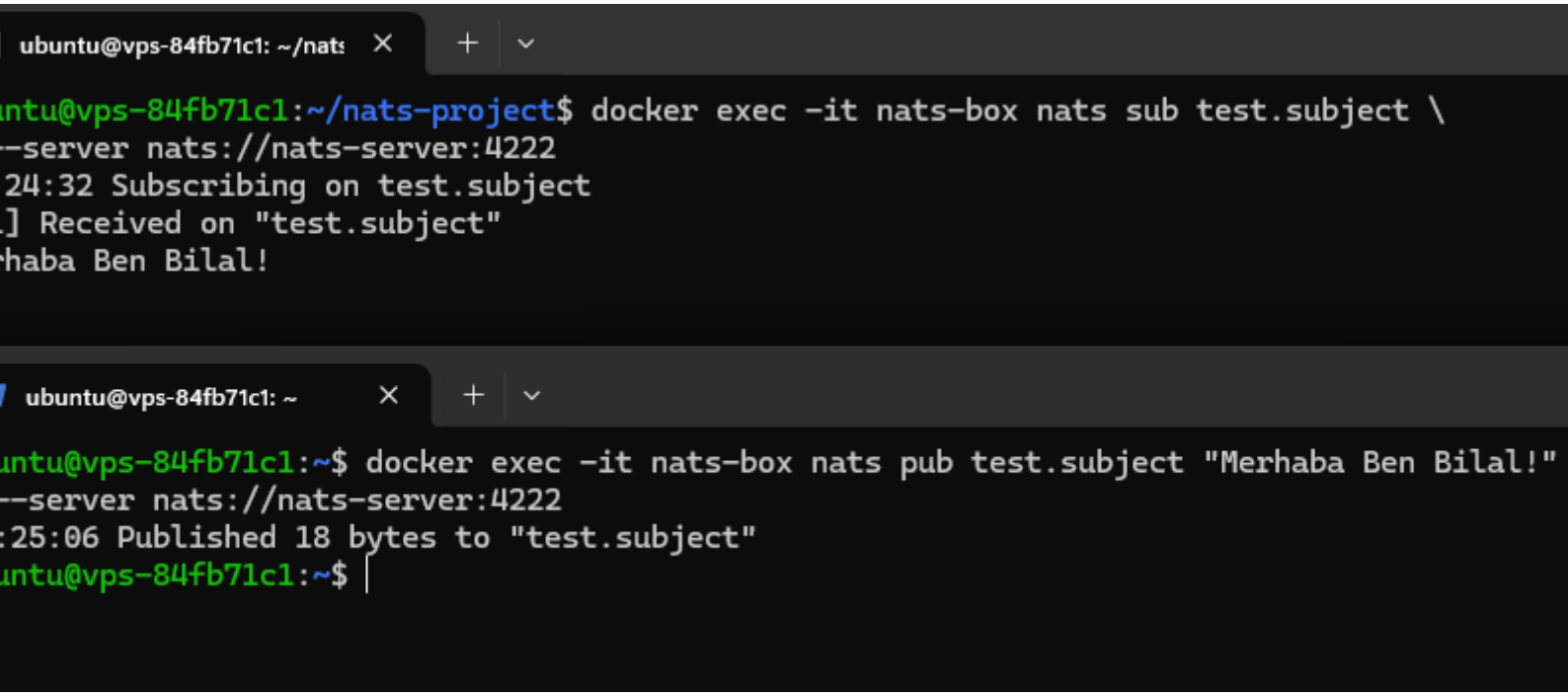
3.3. Test 1: Publish/Subscribe

```
docker exec -it nats-box nats sub test.subject \
  --server nats://nats-server:4222
# 14:23:33 Subscribing on test.subject
```

Listing 6: Subscriber — Terminal 1

```
docker exec -it nats-box nats pub test.subject "Merhaba Ben
    Bilal!" \
  --server nats://nats-server:4222
# 14:23:33 Published 13 bytes to "test.subject"
```

Listing 7: Publisher — Terminal 2



Şekil 3: Pub/Sub Mesaj İletimi

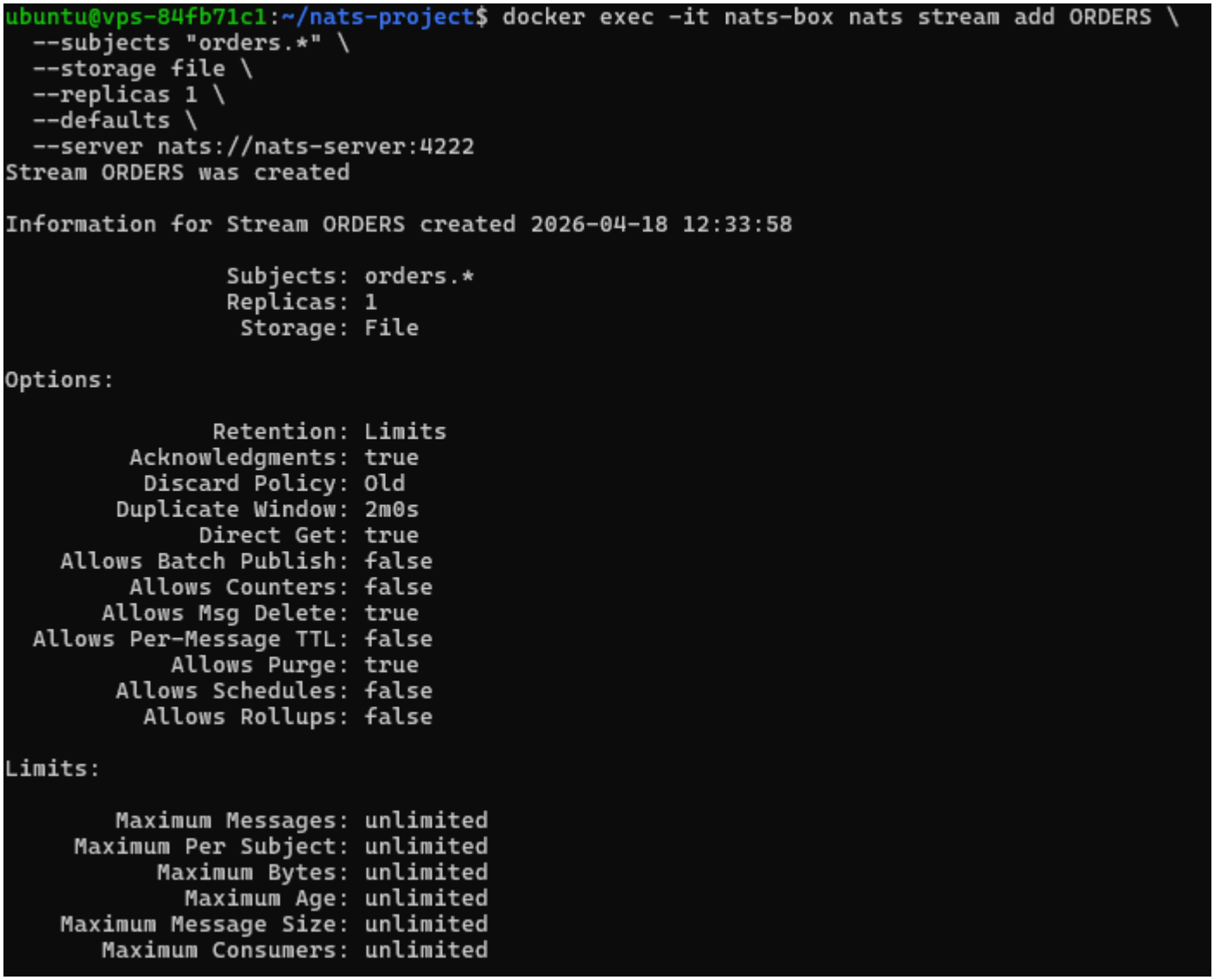
Önemli Not

İlk denemede `--server` parametresi belirtilmediğinde `nats: no servers available for connection` hatası alındı. Docker container ağında DNS çözümlemesi için container adının açıkça belirtilmesi gerektiği anlaşıldı.

3.4. Test 2: JetStream (Kalıcı Mesajlaşma)

```
docker exec -it nats-box nats stream add ORDERS \
  --subjects "orders.*" \
  --storage file \
  --replicas 1 \
  --defaults \
  --server nats://nats-server:4222
# Stream ORDERS was created
# Storage: File | Replicas: 1
```

Listing 8: JetStream Stream Oluşturma



```
ubuntu@vps-84fb71c1:~/nats-project$ docker exec -it nats-box nats stream add ORDERS \
  --subjects "orders.*" \
  --storage file \
  --replicas 1 \
  --defaults \
  --server nats://nats-server:4222
Stream ORDERS was created

Information for Stream ORDERS created 2026-04-18 12:33:58

                Subjects: orders.*
                Replicas: 1
                 Storage: File

Options:

               Retention: Limits
         Acknowledgments: true
          Discard Policy: Old
        Duplicate Window: 2m0s
             Direct Get: true
    Allows Batch Publish: false
         Allows Counters: false
       Allows Msg Delete: true
  Allows Per-Message TTL: false
            Allows Purge: true
        Allows Schedules: false
          Allows Rollups: false

Limits:

        Maximum Messages: unlimited
     Maximum Per Subject: unlimited
           Maximum Bytes: unlimited
             Maximum Age: unlimited
    Maximum Message Size: unlimited
       Maximum Consumers: unlimited
```

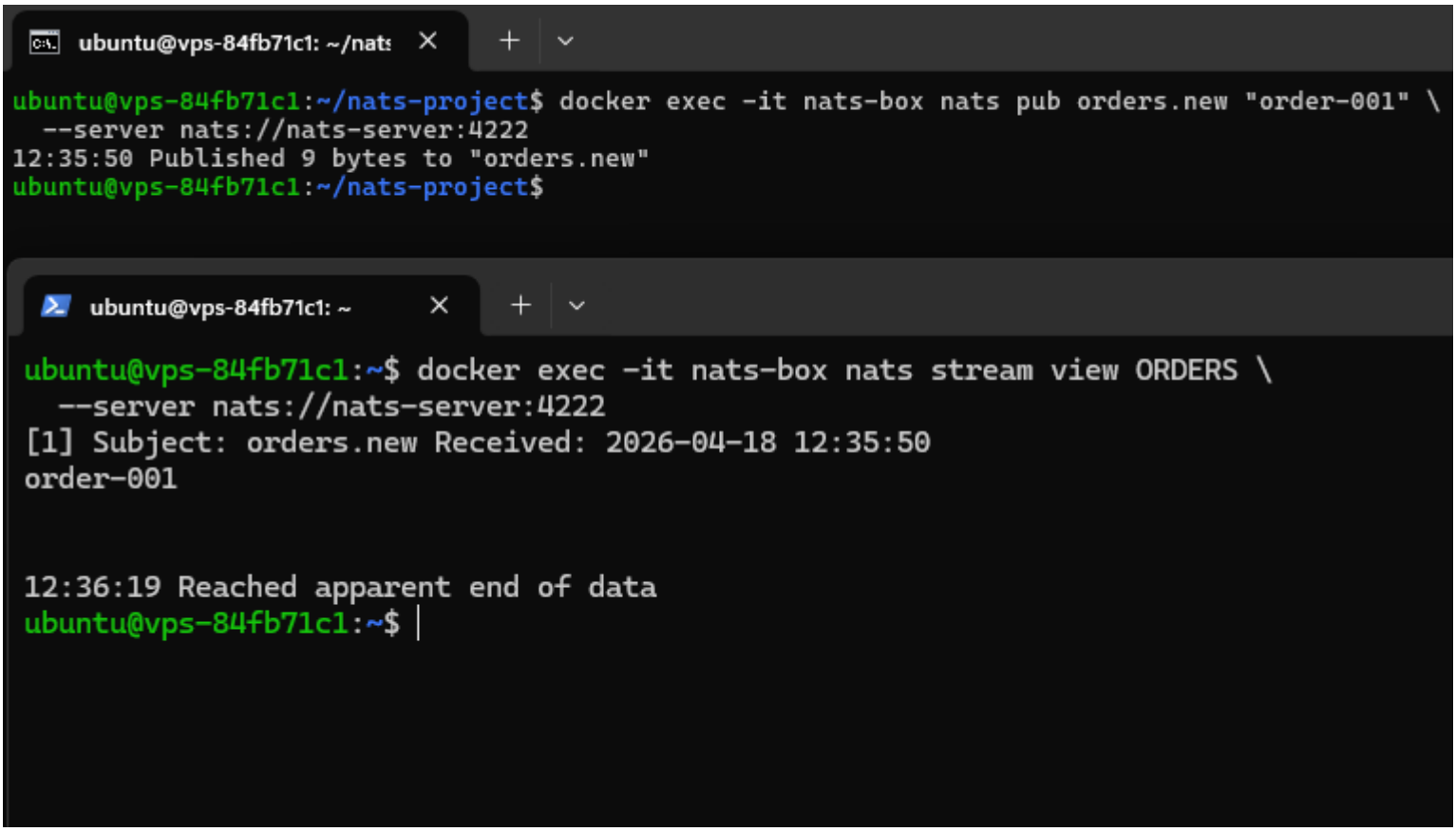
Şekil 4: JetStream Stream Oluşturma

```
# Mesaj gonder
```

```
docker exec -it nats-box nats pub orders.new "order-001" \
  --server nats://nats-server:4222
# 14:24:17 Published 9 bytes to "orders.new"

# Mesaji goruntule
docker exec -it nats-box nats stream view ORDERS \
  --server nats://nats-server:4222
# [1] Subject: orders.new Received: 2026-04-10 14:24:17
# order-001
```

Listing 9: Mesaj Gönderme ve Görüntüleme



Şekil 5: JetStream Kalıcı Mesaj Görüntüleme

3.5. Test 3: Monitoring API

```
curl http://localhost:8222/varz | python3 -m json.tool
```

Listing 10: HTTP Monitoring Endpoint Sorgusu

Testler öncesi ve sonrasında alınan monitoring verileri karşılaştırıldığında belirgin farklılıklar gözlemlenmiştir:

Tablo 4: Monitoring Metrikleri Karşılaştırması (Öncesi / Sonrası)

Metrik	Test Öncesi	Test Sonrası
Uptime	34 saniye	2 dakika 28 saniye
Bellek Kullanımı	11.15 MB	16.39 MB
Toplam Bağlantı	0	5
Gelen Mesaj Sayısı	0	9
Giden Mesaj Sayısı	0	10
JetStream Storage	0 B	49 B
JetStream API Çağrısı	0	5

Temel Bulgu

Bellek kullanımı, 9 mesajın işlenmesinin ardından yalnızca 11.15 MB'tan 16.39 MB'a yükseldi. Toplam artış **5.24 MB**. Bu değer, NATS'ın rakiplerine kıyasla son derece verimli bir bellek yönetimine sahip olduğunu kanıtlamaktadır.

3.6. Test 4: Consumer (Tüketici) Oluşturma

```
# Consumer olustur
docker exec -it nats-box nats consumer add ORDERS CONSUMER1 \
  --deliver all --pull --defaults \
  --server nats://nats-server:4222

# Mesaji oku
docker exec -it nats-box nats consumer next ORDERS CONSUMER1 \
  --server nats://nats-server:4222
# [14:25:26] subj: orders.new / cons seq: 1 / str seq: 1
# order-001
# Acknowledged message
```

Listing 11: Consumer Oluşturma ve Mesaj Tüketme

```
ubuntu@vps-84fb71c1: ~/nats
ubuntu@vps-84fb71c1:~/nats-project$ docker exec -it nats-box nats consumer next ORDERS CONSUMER1 \
  --server nats://nats-server:4222
[12:39:08] subj: orders.new / tries: 1 / cons seq: 1 / str seq: 1 / pending: 0

order-001

Acknowledged message

ubuntu@vps-84fb71c1:~/nats-project$
```

```
ubuntu@vps-84fb71c1: ~
ubuntu@vps-84fb71c1:~$ docker exec -it nats-box nats consumer add ORDERS CONSUMER1 \
  --deliver all --pull --defaults \
  --server nats://nats-server:4222
Information for Consumer ORDERS > CONSUMER1 created 2026-04-18 12:39:03

Configuration:

                    Name: CONSUMER1
               Pull Mode: true
          Deliver Policy: All
              Ack Policy: Explicit
                Ack Wait: 30.00s
           Replay Policy: Instant
         Max Ack Pending: 1,000
       Max Waiting Pulls: 512

State:

            Host Version: 2.12.6
      Required API Level: 0 hosted at level 3
  Last Delivered Message: Consumer sequence: 0 Stream sequence: 0
    Acknowledgment Floor: Consumer sequence: 0 Stream sequence: 0
        Outstanding Acks: 0 out of maximum 1,000
    Redelivered Messages: 0
    Unprocessed Messages: 1
           Waiting Pulls: 0 of maximum 512
ubuntu@vps-84fb71c1:~$
```

Şekil 6: Consumer Mesaj Tüketimi

3.7. Karşılaşılan Sorunlar ve Çözümler

Tablo 5: Karşılaşılan Sorunlar ve Çözümleri

Sorun	Neden	Çözüm
`no servers available`	Container içinde localhost NATS'a ulaşmıyor	`--server nats://nats-server:4222` eklendi
İlk pull'da yavaşlık	İmajlar ilk kez indiriliyor	İkinci çalıştırmada anlık başladı

4. Karşılaştırmalı Değerlendirme

NATS, aynı kategoride öne çıkan iki alternatifle karşılaştırılmıştır: Apache Kafka ve RabbitMQ. Her iki sistem de kurumsal düzeyde yaygın kullanıma sahip olup farklı güçlü yönler barındırmaktadır [5].

4.1. Genel Karşılaştırma

Tablo 6: NATS, Apache Kafka ve RabbitMQ Karşılaştırma Matrisi

Kriter	NATS	Apache Kafka	RabbitMQ
Lisans	Apache 2.0	Apache 2.0	Mozilla MPL 2.0
CNCF Statüsü	**Graduated**	—	—
Yazıldığı Dil	Go	Scala/Java	Erlang
Minimum RAM	**~10 MB**	~512 MB	~128 MB
Harici Bağımlılık	**Yok**	ZooKeeper/KRaft	Erlang Runtime
Protokol	NATS Protokolü	Kafka Protokolü	AMQP, MQTT, STOMP
Kalıcı Mesajlaşma	JetStream	Evet (varsayılan)	Evet (queues)
Max Throughput	**~10M msg/s**	~1M msg/s	~50K msg/s
Gecikme	**<1ms**	2–10ms	1–5ms
Kurulum Zorluğu	**Kolay**	Zor	Orta
Multi-tenancy	Account sistemi	Sınırlı	Virtual Hosts

4.2. Performans Analizi

NATS, mesaj işleme hızında Kafka ve RabbitMQ'yu önemli ölçüde geride bırakmaktadır. Bunun temel nedeni NATS'ın mesaj yönlendirme için hashmap tabanlı O(1)

subject matching kullanmasıdır. Ayrıca NATS'ın Go ile yazılmış olması, düşük bellek tüketimine ve etkin goroutine yönetimine olanak tanır.

Kafka, yüksek hacimli log tabanlı sistemlerde throughput açısından güçlüdür; ancak JVM tabanlı olması nedeniyle bellek kullanımı yüksektir. RabbitMQ ise mesaj routing esnekliği açısından avantajlıdır fakat throughput kapasitesi en düşük olanıdır.

4.3. Topluluk ve Ekosistem

Tablo 7: Topluluk ve Ekosistem Karşılaştırması

Kriter	**NATS**	**Kafka**	**RabbitMQ**
GitHub Stars	∼16K	∼29K	∼12K
CNCF Desteği	Graduated	Yok	Yok
Resmi Client Dili	40+	20+	30+
Aktif Geliştirici	Synadia	Apache/Confluent	Broadcom/VMware
Dokümantasyon	Kapsamlı	Kapsamlı	Orta

4.4. Öğrenme Eğrisi

NATS: Tek binary kurulum, minimal konfigürasyon. Temel pub/sub 10 dakikada öğrenilebilir. JetStream semantiği daha derin anlayış gerektirir.

Kafka: ZooKeeper/KRaft, broker, partition, consumer group, offset gibi kavramların anlaşılması gerekir. Öğrenme eğrisi dikeydir.

RabbitMQ: Exchange, binding, queue, routing key kavramları; AMQP protokolü. Orta düzey öğrenme eğrisi.

4.5. Kullanım Senaryosu Uyumluluğu

Tablo 8: Senaryo Bazlı Uyumluluk Değerlendirmesi

Senaryo	**NATS**	**Kafka**	**RabbitMQ**
IoT / Edge mesajlaşma	**Mükemmel**	Ağır	İyi
Mikro hizmet iletişimi	**Mükemmel**	İyi	İyi
Büyük veri pipeline	Orta	**Mükemmel**	Zayıf
Olay kaydı (event log)	İyi	**Mükemmel**	Orta
Gerçek zamanlı bildirim	**Mükemmel**	Gecikmeli	İyi
Kısıtlı kaynak ortamı	**Mükemmel**	Uygunsuz	Orta

5. Sonuç ve Öneriler

5.1. Güçlü Yönler

1. **Olağanüstü hafiflik:** ∼10 MB bellek ile başlayan NATS, kaynak kısıtlı ortamlarda (IoT gateway, edge node) biçilmiş kaftandır.
2. **Sıfır bağımlılık:** ZooKeeper, Erlang runtime veya benzeri altyapı gerektirmez. Tek binary ile çalışır.
3. **Çok modlu iletişim:** Aynı altyapıda pub/sub, request/reply ve JetStream desteği.
4. **CNCF Graduated:** Üretim ortamı olgunluğu ve uzun vadeli destek güvencesi.
5. **Multi-protocol:** MQTT, WebSocket ve core NATS protokollerini aynı sunucudan destekler.

5.2. Zayıf Yönler

1. **Büyük veri ekosistemi entegrasyonu:** Kafka'nın Flink, Spark gibi araçlarla derin entegrasyonuna kıyasla NATS bu alanda sınırlıdır.
2. **JetStream kısıtlamaları:** Kafka'nın partition tabanlı parallel processing kapasitesine henüz tam olarak ulaşamamaktadır.
3. **Kurumsal destek:** Confluent (Kafka) gibi güçlü ticari desteğe sahip değildir.

5.3. Hangi Senaryolarda NATS Tercih Edilmeli?

- **IoT ve edge computing:** Düşük bellek ve ağ bant genişliği gereksinimleri NATS'ı ideal kılar
- **Mikro hizmet iletişimi:** Hafif, hızlı servisler arası mesajlaşma
- **Gerçek zamanlı sistemler:** Fintech, oyun, bildirim sistemleri
- **Multi-tenant SaaS:** Account sistemi ile izolasyon gereken platformlar

5.4. Öğrenilen Dersler

Bu çalışma boyunca mesajlaşma sistemlerinin bulut-yerel mimarilerdeki kritik rolü somut olarak gözlemlenmiştir. NATS'ın sıfır konfigürasyonla dakikalar içinde çalışır hale gelmesi ve anlık mesaj iletimini <1ms altında gerçekleştirmesi, "*basitlik ve performans*" felsefesinin pratik yansımasıdır.

JetStream özelliğinin, Core NATS'a persistent messaging kapasitesi eklemesi özellikle

dikkat çekicidir: Aynı altyapıda hem geçici hem de kalıcı mesajlaşma mümkün hale gelmektedir. Bu özellik NATS'ı saf pub/sub sistemlerinin ötesine taşımaktadır.

Kaynakça

Önemli Not

Bu çalışmada yapay zekâ araçları (Claude - Anthropic) içerik taslağı ve düzenleme süreçlerinde yardımcı amaçlı kullanılmış; tüm teknik kurulum, testler ve değerlendirmeler bizzat gerçekleştirilmiştir.

Kaynaklar

[1] Synadia Communications. (2024). *NATS Official Documentation*. https://docs.nats.io

[2] Synadia Communications. (2024). *NATS Concepts: Subject-Based Messaging, JetStream*. https://docs.nats.io/nats-concepts/overview

[3] Synadia Communications. (2024). *JetStream Technical Documentation*. https://docs.nats.io/nats-concepts/jetstream

[4] Cloud Native Computing Foundation. (2022). *NATS Graduates within the Cloud Native Computing Foundation*. https://www.cncf.io/projects/nats/

[5] Dobbelaere, P., & Esmaili, K. S. (2017). Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations. *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems (DEBS '17)*. https://doi.org/10.1145/3093742.3093908

[6] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications. ISBN: 978-1617294549.

[7] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), 70–93. https://doi.org/10.1145/2898442.2898444